YILDIZ TECHNICAL UNIVERSITY

FACULTY OF CHEMICAL AND METALLURGICAL ENGINEERING

MATHEMATICAL ENGINEERING

MTM4692 – APPLIED SQL

PROJECT MILESTONE TECHNICAL REPORT

PROJECT NAME : NUTRITRACK

LECTURER : FETTAH KIRAN

STUDENTS : 20058611 ENDER MERT KUTBAY
22058601 BERKAY MUTLU
23052612 KENAN PEHLİVAN

Github: https://github.com/emertk/MTM4692-NutriTrack-SQL

1. Problem Statement

Modern individuals who care about their health are forced to use multiple disconnected apps, one for food logging, one for workouts, one for weight tracking, and one for sleep. None of these talk to each other, and none can answer the question that matters most: am I in a calorie deficit today, accounting for my resting metabolism, my workout, and every meal?

Beyond calorie counting, there is a deeper problem: most apps ignore Basal Metabolic Rate(BMR), the energy your body burns at rest, which changes every time your weight or activity level changes. A static calorie target set at the start of a diet becomes increasingly wrong as weight is lost. Furthermore, manually searching for every food item is tedious and error-prone, leading to inconsistent logging that undermines the entire tracking effort.

NutriTrack addresses these problems with a unified SQLite relational database that:

1) Recalculates BMR and TDEE automatically on every weight update,

2) Enables food entry via barcode scan or AI photo recognition,

3) tracks workouts with MET-based calorie burn calculations, and

4) Integrates water intake, sleep, step count, and body measurements, all in one queryabledata source.

2. Why It Matters

• **Metabolic accuracy:** Static calorie targets fail. Dynamic BMR recalculation ensures targets always reflect the user's current body composition and activity level.

• **Reduced friction**: Barcode scanning and AI image recognition eliminate the biggest barrier to consistent food logging, manual search and entry.

• **Holistic picture:** Sleep, hydration, and step data are correlated with nutrition and exercise in the same database, enabling insights impossible across separate apps (e.g. 'does poor sleep predict overeating the next day?').

• **Progress visibility:** Body measurements stored over time with automatic BMI and waist-to-hip ratio calculations give users objective, quantified progress feedback.

3. Preliminary Schema

Table	Domain	Key Columns	Links To
user	Identity	username, gender, height_cm, bmr_formula, activity_level	—
bmr_log	Metabolism	bmr_kcal, tdee_kcal, formula_used, trigger_event, suggested macros	user
user_goal	Goals	goal_type, daily_calorie_target, macro targets, based_on_bmr_id	user, bmr_log
food_item	Nutrition	barcode, calories_per_100g, protein/carbs/fat/sodium per 100g	—
barcode_scan	Nutrition	barcode, scan_result, api_response_json	user, food_item
ai_image_scan	Nutrition	image_path, estimated_weight_g, confidence_score, user_corrected	user, food_item
meal_log	Nutrition	meal_type, meal_date, total_calories, total_protein_g, total_fat_g	user
meal_log_item	Nutrition	quantity_g, calories, protein_g, entry_source	meal_log, food_item, scans
exercise_type	Exercise	name, category, muscle_groups, met_value, is_strength	—
workout_session	Exercise	session_date, duration_min, total_calories_burned, perceived_effort	user

workout_set	Exercise	set_number, reps, weight_kg, duration_min, calories_burned	workout_session, exercise_type
daily_tracker	Wellness	water_ml, steps, sleep_start/end, sleep_quality, mood	user
body_measurement	Wellness	weight_kg, body_fat_pct, BMI (virtual), waist/hip/arm/thigh cm	user, bmr_log

3.1 BMR / TDEE Calculation Logic

The **bmr_log** table stores a full snapshot of every BMR calculation. A new row is inserted automatically whenever the user logs a new weight, changes their activity level, or updates their goal. Three formulas are supported:

Formula	Equation	Best For
Mifflin-St Jeor	10×kg + 6.25×cm - 5×age + 5	General population, most accurate
Harris-Benedict	13.397×kg + 4.799×cm - 5.677×age + 88.362	Classic, widely referenced
Katch-McArdle	370 + 21.6 × lean_mass_kg	Athletes with known body fat %

TDEE is computed as **BMR × activity multiplier** (1.2 to 1.9). The table also stores three derived calorie targets: Cut = TDEE - 500, Maintain = TDEE, Bulk = TDEE + 300, plus macro suggestions using the 2 g/kg protein rule.

4. Entity Relationships

All foreign keys enforce referential integrity via PRAGMA foreign_keys = ON. The central hub is the user table, every other table references it. Key relationship patterns:

user → bmr_log (1:N) : One user has many BMR snapshots over time. The most recent is retrieved via the *v_latest_bmr* view.

bmr_log → user_goal (1:N) : Each goal records which BMR calculation it was based on (based_on_bmr_id FK), ensuring historical goals remain accurate even as BMR changes.

food_item → meal_log_item (1:N) : One food appears in many meal entries. Macros are calculated per entry as *quantity_g / 100 × per_100g value*.

meal_log → meal_log_item (1:N, CASCADE) : Deleting a meal deletes all its items. Each item optionally references the barcode_scan or ai_image_scan that created it.

workout_session → workout_set (1:N, CASCADE) : Each session contains many sets. Calories per set: *MET × user_weight_kg × (duration_min / 60)*.

body_measurement → bmr_log (N:1) : Each measurement can trigger a new BMR recalculation, linked via triggered_bmr_id.

daily_tracker has a UNIQUE(user_id, track_date) constraint, one row per user per day.

5. Proposed SQL Features

Feature	Where Used
LEFT JOIN, gap detection.	90-day calendar vs meal_log: find untracked days
CROSS JOIN	Calendar × user for full grid
Correlated subquery	Each patient's most recent body measurement
GROUP BY + HAVING	Doctors with > 2 completed sessions; top foods by frequency
Aggregate: SUM, AVG, COUNT, ROUND	Weekly macro averages vs goal targets
CREATE VIEW (4 views)	v_latest_bmr, v_daily_nutrition_summary, v_goal_progress…
CREATE INDEX (20 indexes)	Barcode, datetime, status, FK columns
EXPLAIN QUERY PLAN	Barcode lookup index verification
WITH, single CTE	Monthly calorie summary, food frequency ranking

WITH, multiple chained CTEs	Comprehensive user health summary (5 CTEs)
WITH RECURSIVE, calendar	90-day date series for gap analysis
WITH RECURSIVE, projection	12-week weight projection toward goal
Window: LAG, FIRST_VALUE	BMR trend, weight change vs first measurement

6. Sample Queries

Query 1) Dynamic BMR/TDEE History with Change Tracking

Uses a window function (LAG) to show how BMR and TDEE shift with each weight update:

```
SELECT u.username, bl.calculated_at,
      bl.trigger_event, bl.weight_kg,
      bl.bmr_kcal, bl.tdee_kcal, bl.bmr_kcal -
      LAG(bl.bmr_kcal)
   OVER (PARTITION BY bl.user_id ORDER BY bl.calculated_at) AS bmr_change,

      bl.tdee_kcal - LAG(bl.tdee_kcal)
         OVER (PARTITION BY bl.user_id ORDER BY bl.calculated_at) AS
tdee_change FROM  bmr_log bl JOIN user u ON u.user_id = bl.user_id
ORDER  BY u.user_id, bl.calculated_at;
```

Query 2) Daily Calorie Balance (Multi-CTE)

Combines nutrition, exercise, and BMR data into a single daily balance figure:

```
WITH daily_in AS (
    SELECT user_id, meal_date, SUM(total_calories) AS cal_in
    FROM   meal_log GROUP BY user_id, meal_date
),
daily_burned AS (
    SELECT user_id, session_date, SUM(total_calories_burned) AS burned
```

```
    FROM   workout_session GROUP BY user_id, session_date
),
latest_bmr AS (
    SELECT user_id, tdee_kcal FROM v_latest_bmr
)

SELECT u.username, di.meal_date,
       ROUND(di.cal_in, 0)                              AS calories_in,
       ROUND(COALESCE(db.burned, 0), 0)                 AS exercise_burned,
       ROUND(lb.tdee_kcal, 0)                           AS bmr_tdee,
       ROUND(di.cal_in - lb.tdee_kcal
             - COALESCE(db.burned, 0), 0)               AS net_balance
FROM   daily_in di
JOIN   user u ON u.user_id = di.user_id
JOIN   latest_bmr lb ON lb.user_id = di.user_id
LEFT JOIN daily_burned db ON db.user_id = di.user_id
                          AND db.session_date = di.meal_date
ORDER  BY u.username, di.meal_date;
) SELECT week_num, proj_date, proj_weight, ROUND(proj_weight -
target_weight_kg, 2) AS kg_remaining FROM projection;
```

Query 3) 12-Week Weight Projection (Recursive CTE)

Projects future weight week-by-week based on the user's active weekly change goal:

```
WITH RECURSIVE projection AS (
   SELECT 1 AS week_num, date('now') AS proj_date,
         bm.weight_kg AS proj_weight,
         g.target_weight_kg
   FROM  body_measurement bm
   JOIN  user_goal g ON g.user_id = bm.user_id AND
   g.is_active = 1 WHERE bm.user_id = 1
     AND  bm.measured_date = (SELECT MAX(measured_date)
                          FROM body_measurement WHERE user_id = 1)
   UNION ALL
```

```
SELECT week_num + 1,
      date(proj_date, '+7 days'),
      ROUND(proj_weight + (SELECT weekly_weight_change_kg
                        FROM user_goal WHERE user_id
                        = 1 AND is_active = 1), 2),
      target_weight_kg
FROM  projection
WHERE  week_num < 12
  AND  ABS(proj_weight - target_weight_kg) > 0.
```

7. Project Scope & Feasibility

Dimension	**Target**
Tables	13 — 4 domains: Identity, Nutrition, Exercise, Wellness
Sample data	194+ rows across all tables, realistic names, values, timestamps
Key queries	16 queries covering all MTM4692 Weeks 1–7 concepts
Views	6 views: v_latest_bmr, v_daily_nutrition_summary, v_goal_progress, v_body_progress, v_weekly_progress, v_daily_calorie_balance
Indexes	20 indexes on barcode, datetime, status, and FK columns
CTEs	Single, multi-chain, recursive calendar, recursive projection
Database	SQLite 3, no external server required, PRAGMA foreign_keys = ON
Deliverable	schema.sql + seed.sql + queries.sql + README.md on public GitHub

The 13-table scope covers all required relationships and SQL techniques without over-engineering. Every table serves a distinct real-world purpose — there is no padding. The project is fully implementable in SQLite with no external dependencies, and all SQL files have been tested and verified to run with zero error